

MOZI Hash function

Submission to ICCS NGCC

Wenquan Bi Patrick Derbez Xiaoyang Dong*
Pierre-Alain Fouque Keting Jia Lingyue Qin
André Schrottenloher Liyuan Tang
Boxin Zhao

*Corresponding submitter: Xiaoyang Dong
Tsinghua University, Beijing, P.R.China
xiaoyangdong@tsinghua.edu.cn

Tsinghua University, Beijing, P.R.China
Zhongguancun Laboratory, Beijing, P.R.China
Univ Rennes, Inria, CNRS, IRISA, Rennes, France

September 28, 2026

Contents

1	Introduction	3
2	Specification	3
2.1	Notations	3
2.2	Parameters	3
2.3	Sponge function and Feed-forward Sponge	4
2.4	The Permutations	6
2.4.1	Specification of S-box	7
2.4.2	Specification of MixColumn	7
2.4.3	Specification of ShiftRow	9
2.4.4	Specification of AddConstant	10
2.5	Domain separation	11
3	Security Claims	11
4	Security Analysis	11
4.1	Theoretical Security Analysis	11
4.2	Practical Security Analysis	12
4.3	Basic Properties	12
4.3.1	Differential and Linear Analysis	12
4.3.2	Meet-in-the-middle Attack	12
4.4	Quantum Analysis	13
4.5	Randomness Analysis	14
4.6	Resistance to Length Extension Attacks	15
5	Features	15
6	Design Rationale	16
6.1	Choice of the S-box Operation	16
6.2	Choice of Mixcolumn Operation	16
6.3	Choice of ShiftRow Operation	16
7	Implementation	17
7.1	Bit-Slice and Parallel Implementation	17
7.1.1	S-boxes	18
7.1.2	MixColumns	18
7.1.3	ShiftRows	19
7.2	SIMD Accelerated Implementation	19
7.3	Resource-constrained Implementation	19
7.4	Software Implementation Performance	20
7.5	Hardware	21
A	Table	24
B	Randomness Analysis Result	24

1 Introduction

MOZI, often known as the an eminence ancient Chinese inventor, is a family of hash functions based on the 2048-bit permutation. It is designed to achieve a balance between hardware cost and software efficiency, while providing clear lower bounds for differential/linear active Sboxes and facilitating a clear understanding of the security against various attacks. MOZI works in two modes of operations, *i.e.*, the Sponge function [4] and the feed-forward Sponge [11, 17].

2 Specification

2.1 Notations

a/b	number of rounds of the permutation, $a = 20$ and $b = 24$
r	size of rate or outer part in bit
c	size of capacity or inner part in bit
h	sizes of the digest in bits
$\text{MSB}_s(x)$	the most significant s bits of x
$\text{LSB}_s(x)$	the least significant s bits of x

2.2 Parameters

MOZI is a family of hash functions, which has 4 variants with digest bitsize $h=384, 512, 768$, and 1024 bits, denoted as (MOZI-384, MOZI-512, MOZI-768, and MOZI-1024). Besides, two extendable output functions (XOFs) to produce hash outputs of arbitrary length are defined as MOZI-384-XOF and MOZI-512-XOF.

- For MOZI-384 and MOZI-512, the hash functions are built on the Sponge function [4], which is defined as $\text{SPONGE}_{h,r,a}(M, \ell)$ parameterized by the digest length h , the rate r , the number of rounds a of the permutation p^a (p is a single round of the permutation), the input message M and output bitsize ℓ (if $h \neq 0, \ell = h$). $\text{SPONGE}_{h,r,a}(M, \ell)$ maps an input message M of arbitrary length to a fixed-length hash output T :

$$\text{SPONGE}_{h,r,a}(M, \ell) = T, \text{ where } |T| = h \text{ bits.} \quad (1)$$

- For MOZI-768 and MOZI-1024, the hash functions are built on the feed-forward Sponge function [11], which is defined as $\text{F-SPONGE}_{h,r,b}(M)$ parameterized by the digest length h , the rate r , the number of rounds b of the permutation p^b (p is a single round of the permutation), the input message M . $\text{F-SPONGE}_{h,r,b}(M)$ maps an input message M of bitsize smaller than 2^{64} to a fixed-length hash output T :

$$\text{F-SPONGE}_{h,r,b}(M) = T, \text{ where } |T| = h \text{ bits.} \quad (2)$$

- For MOZI-384-XOF and MOZI-512-XOF, the sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$ is applied with $h = 0$. The output is of arbitrary bitsize ℓ .

$$\text{SPONGE}_{0,r,a}(M, \ell) = T, \text{ where } |T| = \ell \text{ bits.} \quad (3)$$

Table 1 summarizes the recommended instances for hash functions and and XOF with specified parameters.

Table 1: Parameters for recommended hashing functions and XOFs

Name	Algorithm	Bit size of			state	Rounds
		digest (h)	capacity(c)	rate (r)		
MOZI-384	$\text{SPONGE}_{384,1280,20}(M, \ell)$	384	768	1280	2048	20
MOZI-512	$\text{SPONGE}_{512,1024,20}(M, \ell)$	512	1024	1024	2048	20
MOZI-768	$\text{F-SPONGE}_{768,1216,24}(M)$	768	832	1216	2048	24
MOZI-1024	$\text{F-SPONGE}_{1024,960,24}(M)$	1024	1088	960	2048	24
MOZI-384-XOF	$\text{SPONGE}_{0,1280,20}(M, \ell)$	arbitrary	768	1280	2048	20
MOZI-512-XOF	$\text{SPONGE}_{0,1024,20}(M, \ell)$	arbitrary	1024	1024	2048	20

2.3 Sponge function and Feed-forward Sponge

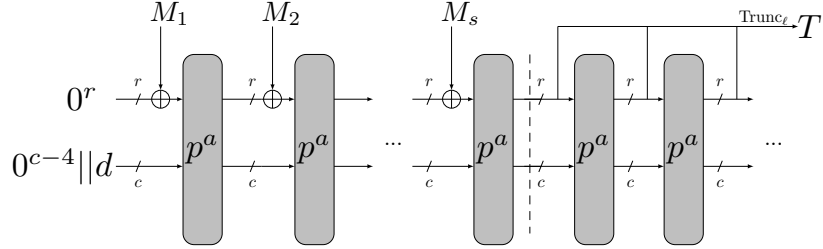


Figure 1: Sponge function

Sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$. The Sponge structure is given in Figure 1 and $\text{SPONGE}_{h,r,a}(M, \ell)$ is specified in Algorithm 1. In the initialization phase, four-bit domain encoding number d is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 2.5.

Algorithm 1: The Sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$

Input: Message $M \in \{0, 1\}^*$, output bitsize $\ell \leq h$ or arbitrary if $h = 0$
Output: Digest $T \in \{0, 1\}^\ell$
 /* Initialization, 4-bit domain encoding d is specified in Table 5 for different ℓ */
 1 $\mathcal{I} \leftarrow 0^{2044} \| d$
 /* Absorbing */
 2 $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$, where the bitsize of M_i ($1 \leq i \leq s$) is r bits
 3 **for** $i = 1$ **to** s **do**
 4 $\mathcal{I} \leftarrow p^a(\mathcal{I} \oplus (M_i \| 0^{2048-r}))$
 /* Squeezing */
 5 $\lambda \leftarrow \lfloor \frac{\ell}{r} \rfloor$
 6 $Z \leftarrow \text{MSB}_r(\mathcal{I})$
 7 **for** $i = 1$ **to** λ **do**
 8 $\mathcal{I} \leftarrow p^a(\mathcal{I})$
 9 $Z \leftarrow Z \| \text{MSB}_r(\mathcal{I})$
 10 **return** $T = \text{MSB}_\ell(Z)$

Feed-forward Sponge Function $\text{F-SPONGE}_{h,r,b}(M)$. The feed-forward sponge construction [17, 11] is given in Figure 2, and $\text{F-SPONGE}_{1024,960,24}(M)$ is specified in Algorithm 2. In the initialization phase, four-bit domain encoding number d is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 2.5.

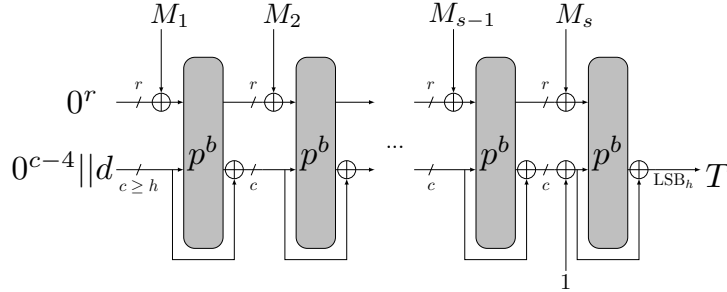


Figure 2: Feed-forward Sponge function

Algorithm 2: The feed-forward Sponge function $\text{F-SPONGE}_{h,r,b}(M)$

Input: Message $M \in \{0,1\}^*$
Output: Digest $T \in \{0,1\}^h$
 /* Initialization, 4-bit domain encoding d is specified in Table 5 for different h */
 1 $\mathcal{I} \leftarrow 0^{2044} \| d$
 /* Absorbing */
 2 $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$, where the bitsize of M_i ($1 \leq i \leq s$) is r bits
 3 **for** $i = 1$ **to** s **do**
 4 **if** $i \neq s$ **then**
 5 $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 0^c)$
 6 **else**
 7 $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 0^{c-1} \| 1)$
 8 $\mathcal{I} \leftarrow (0^r \| \text{LSB}_c(\mathcal{I})) \oplus p^b(\mathcal{I})$
 /* Squeezing */
 9 $T \leftarrow \text{LSB}_h(\mathcal{I})$
 10 **return** T

2.4 The Permutations

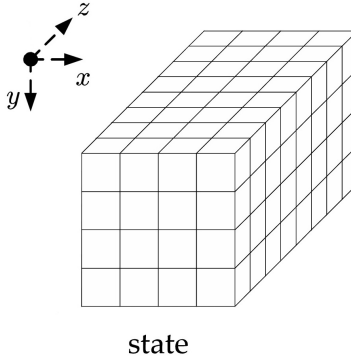


Figure 3: Parts of the State Array

The main component of MOZI is the 2048-bit permutation p^a or p^b . The permutation operates on a state $\mathcal{I}$ that is a three-dimensional array of elements of $\text{GF}(2)$, namely $\mathcal{I}[4][4][128]$. The expression $\mathcal{I}[x, y, z]$, with $0 \leq x, y \leq 3$ and $0 \leq z \leq 127$, denotes the bit in position (x, y, z) . The state array is illustrated in Fig. 3 for the case $0 \leq z \leq 7$.

The state bits are indexed in a column-wise lane order. More precisely, for

an external bit index i , the corresponding state bit is $\mathcal{I}[x, y, z]$, where

$$i = (4x + y) \cdot 128 + z, \quad 0 \leq x, y \leq 3, \quad 0 \leq z \leq 127.$$

This ordering is used when the rate part of the state is filled during absorbing and read during squeezing.

The permutation p^a denotes that a round functions p are applied. Each round p applies four operations in sequence: the **S-box**, **MixColumn**, **ShiftRow** and **AddConstant**. The specifications of these operations are provided below.

Algorithm 3: The Permutation p^a

Input: State $\mathcal{I}$, number of rounds a
Output: State $\mathcal{I}$

```

1  $A \leftarrow \mathcal{I}$ 
2 for  $i = 0$  to  $a - 1$  do
    /* S-box */
3   for  $y = 0$  to 3 do
4      $B[3, y] \leftarrow (A[3, y] \& A[2, y]) \oplus A[1, y]$ 
5      $B[1, y] \leftarrow (A[2, y] \& A[1, y]) \oplus A[0, y]$ 
6      $B[0, y] \leftarrow (B[3, y] \& A[0, y]) \oplus A[3, y]$ 
7      $B[2, y] \leftarrow (B[1, y] \& A[3, y]) \oplus A[2, y]$ 
    /* MixColumn */
8    $C \leftarrow \text{MDS}(B)$ 
    /* ShiftRow */
9   for  $x = 0$  to 3 do
10    for  $y = 0$  to 3 do
11       $D[x, y] \leftarrow \text{RotR}(C[x, y], \rho_y^i)$ 
    /* AddConstant */
12    $A \leftarrow D \oplus RC[i]$ 
13  $\mathcal{I} \leftarrow A$ 
14 return  $\mathcal{I}$ 
```

2.4.1 Specification of S-box

Fig. 4 shows both the S-box illustration and its bitslice implementation for MOZI's S-box. At this step, the four highlighted bits form a row, which is taken as the input to a 4-bit S-box, and the process can be formally described as follows:

$$B[\cdot, y, z] = \text{SB}(A[\cdot, y, z])$$

The lookup table of MOZI's S-box is shown in Table 2.

2.4.2 Specification of MixColumn

At this step, each row is regarded as a variable over $\mathbb{F}_2^4$. For every slice, the MixColumn operation is performed by left-multiplying with an MDS matrix M , as illustrated in Fig. 6. Specifically, we have

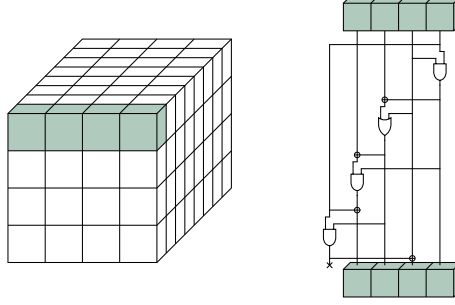


Figure 4: Illustration of S-box

Table 2: The lookup table of S-box

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
SB(x)	0	8	6	13	5	15	7	12	4	14	2	3	9	1	11	10

$$\begin{bmatrix} C[\cdot, 0, z] \\ C[\cdot, 1, z] \\ C[\cdot, 2, z] \\ C[\cdot, 3, z] \end{bmatrix} = M \cdot \begin{bmatrix} B[\cdot, 0, z] \\ B[\cdot, 1, z] \\ B[\cdot, 2, z] \\ B[\cdot, 3, z] \end{bmatrix}.$$

We used the MDS matrix over $\text{GF}(2^4)$ from [9] which is also used in [6], where the modulus polynomial is $x^4 + x + 1$. The MDS matrix is as follows:

$$M \cdot \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} \alpha^2(a) \oplus \alpha^2(b) \oplus \alpha(b) \oplus c \oplus d \\ a \oplus \alpha(b) \oplus b \oplus \alpha^2(c) \oplus c \oplus \alpha^2(d) \oplus \alpha(d) \oplus d \\ a \oplus b \oplus \alpha^2(c) \oplus \alpha^2(d) \oplus \alpha(d) \\ \alpha^2(a) \oplus a \oplus \alpha^2(b) \oplus \alpha(b) \oplus b \oplus c \oplus \alpha(d) \oplus d \end{bmatrix}$$

where α transforms the four bits $x_0, \dots, x_3$ of each nibble by the following multiplication

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

The transformation can be implemented efficiently as depicted in Fig. 5 which is original from [9]. There are 38 XOR operations for this MDS layer. The MDS operation is given formally in Algorithm 4.

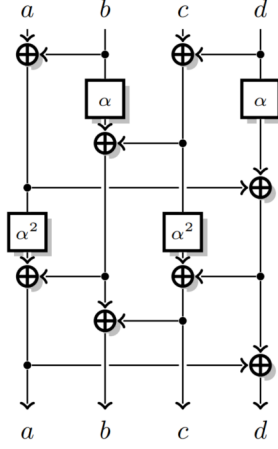


Figure 5: MDS matrix from [9] and [6]

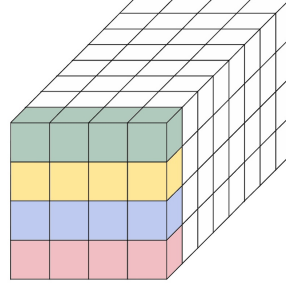


Figure 6: Illustration of MixColumn

Algorithm 4: MDS function $\text{MDS}(B)$

Input: B

Output: C

```

1 Function  $\text{MUL}(S = (x_0, x_1, x_2, x_3))$ :
2    $tmp \leftarrow x_3, x_3 \leftarrow x_2, x_2 \leftarrow x_1, x_1 \leftarrow x_0, x_0 \leftarrow tmp, x_1 \leftarrow x_1 \oplus x_0$ ;
3   return  $(x_0, x_1, x_2, x_3)$ ;

4  $(S_0, S_1, S_2, S_3) \leftarrow (B[0-3, 0], B[0-3, 1], B[0-3, 2], B[0-3, 3])$ 
5  $S_2 \leftarrow S_2 \oplus S_3, S_0 \leftarrow S_0 \oplus S_1$ ;
6  $S_1 \leftarrow \text{MUL}(S_1), S_3 \leftarrow \text{MUL}(S_3)$ ;
7  $S_1 \leftarrow S_1 \oplus S_2, S_3 \leftarrow S_3 \oplus S_0$ ;
8  $S_0 \leftarrow \text{MUL}(S_0), S_2 \leftarrow \text{MUL}(S_2)$ ;
9  $S_0 \leftarrow \text{MUL}(S_0), S_2 \leftarrow \text{MUL}(S_2)$ ;
10  $S_2 \leftarrow S_2 \oplus S_3, S_0 \leftarrow S_0 \oplus S_1$ ;
11  $S_1 \leftarrow S_1 \oplus S_2, S_3 \leftarrow S_3 \oplus S_0$ ;
12  $(C[0-3, 0], C[0-3, 1], C[0-3, 2], C[0-3, 3]) \leftarrow (S_0, S_1, S_2, S_3)$ 
13 end
```

2.4.3 Specification of ShiftRow

Each row is moved along the z direction by several bits, which depends on the fixed SR constants in the round i .

$$D[x, y, z] = \text{RotR}(C[x, y, z], \rho_y^i) = C[x, y, z - \rho_y^i \bmod 128], \quad 0 \leq x, y \leq 3, \quad 0 \leq z \leq 127. \quad (4)$$

An example of SR constants in round $i = 0$ is $\rho_0^0 = 0, \rho_1^0 = 1, \rho_2^0 = 2, \rho_3^0 = 3$ is given in Fig. 7 below.

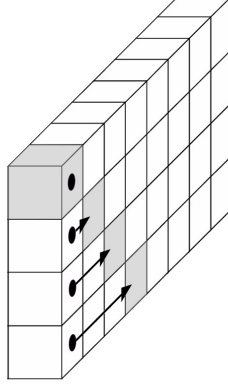


Figure 7: Illustration of ShiftRow

The round function employs different rotation constants depending on the round number i and $0 \leq y \leq 3$, as shown in Table 3.

Table 3: Values of ρ_y^i used in round i ($i \geq 0$) of the permutation

	$y = 0$	$y = 1$	$y = 2$	$y = 3$
$i \bmod 4 = 0$	0	14	20	22
$i \bmod 4 = 1$	0	13	68	91
$i \bmod 4 = 2$	0	27	42	106
$i \bmod 4 = 3$	0	32	48	80

2.4.4 Specification of AddConstant

In round i , the round constant is added to the first byte of the last lane, namely the least significant byte of $D[3, 3]$ under the little-endian convention. That is, $RC[i]$ is XORed to the LSB of $D[3, 3]$ in each round. The round constants are truncated from the value of π as one-byte word for each round. The details of round constants are in Table. 2.4.4.

Round i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$RC[i]$	0x24	0x3F	0x6A	0x88	0x85	0xA3	0x08	0xD3	0x13	0x19	0x8A	0x2E	0x03	0x70	0x73	0x44
Round i	16	17	18	19	20	21	22	23								
$RC[i]$	0xA4	0x09	0x38	0x22	0x29	0x9F	0x31	0xD0								

Table 4: Round constants derived from the hexadecimal digits of π .

2.5 Domain separation

We use the last four bits in IV to make the domain separation. That is, in the initialization phase, the initial state is assigned by concatenating 2044 bits of zeros with the 4-bit domain encoding, which is located in the least significant 4 bits in capacity. The domain encoding for different versions is listed in Table 5. Different versions have different domain encode numbers fed in the last four bits of IV . For example, the input IV value is 2044 bit zeros, concatenating 4 bits 0001 for MOZI-384 in the initialization phase, while for MOZI-384-XOF, the four bit number is 1001.

Table 5: Domain encoding for different hash and XOF versions

Versions	MOZI-384	MOZI-512	MOZI-768	MOZI-1024	MOZI-384-XOF	MOZI-512-XOF
Domain values	0001	0011	0100	0101	1001	1011

3 Security Claims

We provide our security claims for the different variants of MOZI in Table 6. Basically, for all versions of MOZI, we claim that it is hard to find a collision with substantially less than $2^{h/2}$ hash evaluations, and hard to find a (second-)preimage with substantially less than 2^h hash evaluations, where h denotes the digest size.

Table 6: Security claims for recommended parameters of MOZI. For second preimage security spre., security is considered for challenge messages with up to 2^{64} blocks (after padding).

Requirement	Security in bits					
	MOZI-384	MOZI-512	MOZI-768	MOZI-1024	MOZI-384-XOF	MOZI-512-XOF
Collision	192	256	384	512	$\min(384, \ell/2)$	$\min(512, \ell/2)$
Preimage	384	512	768	1024	$\min(384, \ell)$	$\min(512, \ell)$
2nd preimage	384	512	768	1024	$\min(384, \ell)$	$\min(512, \ell)$

4 Security Analysis

4.1 Theoretical Security Analysis

The mode of operation of MOZI-384, MOZI-512, MOZI-384-XOF, and MOZI-512-XOF is based on the sponge methodology proposed by Bertoni et al. [4] and profits from the extensive literature on Sponges, particularly the results on its indistinguishability up to about $2^{c/2}$ calls to the permutation or its inverse, where c is the capacity in bits [4].

The mode of operation of MOZI-768, MOZI-1024 is based on the feed-forward sponge methodology proposed by Guo et al. [11] and Sun et al. [17] with formal security proofs.

4.2 Practical Security Analysis

The practical security of MOZI is ultimately reduced to the security of the permutation it used. In terms of the primary differential cryptanalysis, MOZI have a sufficient number of active S-boxes to guarantee the security against differential cryptanalysis. In the following, we give some concrete analysis like meet-in-the-middle attacks targeting collision attacks and preimage attacks etc.

4.3 Basic Properties

In this section, we give some known properties of the S-box used in MOZI. Tab. 13 shows the differential probabilities corresponding to input and output differences. As can be seen in the table, the maximum differential probability of the S-box is 2^{-2} and its differential branch number is 2. Tab. 14 shows the biases of the linear approximation defined by corresponding input and output masks. The maximum linear probability of the S-box is 2^{-2} and its linear branch number is 2.

Let x_0, x_1, x_2, x_3 and y_0, y_1, y_2, y_3 be the 4-bit input and output of the S-box, where x_0 refers to the most significant bit of the S-box. Then the algebraic normal form (ANF) of the S-box is given by:

$$\begin{aligned} y_0 &= x_3x_2x_0 + x_1x_0 + x_3 \\ y_1 &= x_2x_1 + x_2 + x_1 + x_0 \\ y_2 &= x_3x_0 + x_2x_1 + x_1 \\ y_3 &= x_3x_2 + x_1 \end{aligned}$$

The algebraic degree of the S-box is 3.

4.3.1 Differential and Linear Analysis

The design of MOZI permutation provides clear lower bounds for the number of differential/linear active Sboxes, and a clear understanding of the security against various attacks. In this section, we prove the minimum numbers of the differential/linear active Sboxes with the MILP models. The results are given in Table 7. Therefore, the probability of all 12-round differential characteristics is smaller than $2^{-2} \times 1192 = 2^{-2384}$, which will invalid any collision attacks based on differential attacks.

4.3.2 Meet-in-the-middle Attack

The MOZI permutation achieves full diffusion with 5 rounds, which guarantees that any bits non-linearly affects all bits after 5 rounds in the forward direction.

Table 7: Minimum Numbers of Active Sbox for Different Rounds of MOZI Permutation

Rounds	7	8	9	10	11	12
Minimum Numbers of Active Sbox	169	370	571	665	753	1192

According to Qin *et al.*'s MitM attack model on Sponge function [14] and the partial matching principle [15], the possible number of rounds for partial matching is $(5-1)=4$ rounds. The number of possible rounds for the initial structure is $(5-1)=4$ rounds. Then, the possible number of rounds used for the MitM attack is estimated as $(5-1)+(5-1)+1=9$ rounds for MOZI-384, MOZI-512, MOZI-384-XOF, and MOZI-512-XOF that based on the sponge, leaving a security margin of $20-9=11$ rounds.

For MOZI-768, MOZI-1024 based on on the feed-forward sponge, we evaluate the security against MitM attack following a framework of splice-and-cut technique [15], which is also adopted by the self-evaluation of SKINNY [2] and MIDORI [1]. The possible number of rounds for the partial matching [15] is estimated as $(5-1)+(5-1)+1=9$ rounds. The number of possible rounds for the initial structure is $(5-1)=4$ rounds. The splice-and-cut may extend the number of attack rounds up to the smaller number of full diffusion rounds minus one. Then, possible number of rounds used for MitM is $9+4+4=17$ rounds, leaving a security margin of $24-17=7$ rounds.

We search for MitM trails with the automatic tools of the meet-in-the-middle technique [14, 8]. We get a 7-round MitM trails for sponge-based constructions (i.e., MOZI-384, MOZI-512, MOZI-384-XOF, and MOZI-512-XOF), and 15-round MitM trails for feed-forward sponge based constructions (i.e., MOZI-768, MOZI-1024). The results are listed in Table 8.

Table 8: MitM trails

	Sponge	FF-Sponge
Number of Rounds	7 rounds	15 rounds

4.4 Quantum Analysis

Given a hash function with h -bit digest, the quantum preimage or 2nd preimage attacks should be not efficient than the Grover's algorithm [10], whose time complexities is $2^{h/2}$. At CRYPTO 2022, Schrottenloher and Stevens Marc[16] proposed the quantum preimage attack by introducing the quantum MitM attack. The quantum attack leverages the valid MitM trails. However, as analyzed in Sect. 4.3.2, no MitM trails cover the full rounds of MOZI permutation. Therefore, there is no quantum MitM preimage attack on full MOZI.

The complexity of generic quantum collision-finding algorithms is highly dependent on the resources available to the adversary. Existing literature typically considers three scenarios, distinguished by the type and scale of memory and computing resources, each leading to different time–space tradeoffs.

- **Scenario 1.** Assuming a polynomial-size quantum computer with access to exponentially large quantum random access memory (qRAM), the BHT algorithm [5] achieves both time and qRAM complexity $2^{h/3}$. While this setting yields optimal time complexity for generic quantum collision finding, the requirement for large-scale qRAM poses significant practical challenges.
- **Scenario 2.** In the absence of exponentially large quantum or classical memory, the quantum analogue of the parallel rho method [18, 3, 12] employs S quantum processors to achieve a time-space tradeoff of $2^{h/2}/S$, making it suitable for memory-constrained settings with parallel quantum resources.
- **Scenario 3.** Allowing large classical memory while avoiding exponentially large qRAM, the CNS algorithm [7] finds collisions in time $2^{2h/5}$ using $2^{h/5}$ classical RAM, providing a balanced tradeoff between efficiency and resource availability.

Given a differential characteristic with probability of 2^{-x} , one could find a conforming pair with the Grover’s algorithm in time $2^{x/2}$. Therefore, to be not better than the above three quantum generic attacks, $2^{x/2}$ should be larger than $\max\{2^{h/3}, 2^{h/2}, 2^{2h/5}\}$, i.e., $x > h$. According to the differential analysis of Sect. 4.3.1, the probability of all 12-round differential characteristics is smaller than $2^{-2} \times 1192 = 2^{-2384}$, i.e., the smallest possible x is at least 2384 for 12-round differential trail, and $2384 > h = 512$ or $2384 > h = 1024$. Therefore, the full-round MOZI is security against any dedicated quantum collision attacks.

4.5 Randomness Analysis

We evaluate the randomness of the outputs using 15 statistical tests, including *MonoBitFrequencyTest*, *FrequencyWithinBlockTest*, *PokerTest*, *OverlappingTemplateMatchingTest*, *RunsTest*, *RunsDistributionTest*, *LongestRunOfOnesInABlockTest*, *BinaryDerivativeTest*, *AutocorrelationTest*, *MatrixRankTest*, *CumulativeTest*, *ApproximateEntropyTest*, *LinearComplexityTest*, *MaurerUniversalTest*, and *DiscreteFourierTransformTest*. For each test, the significance level is set to $\alpha = 0.01$. A sequence is considered to pass a test if the corresponding p -value is greater than or equal to α . The detailed randomness analysis results are given in Section B.

Our experimental results show that, for all input types (random, high Hamming weight, and low Hamming weight), the outputs consistently satisfy this criterion across all tests. Consequently, the outputs are shown to pass all statistical randomness tests.

4.6 Resistance to Length Extension Attacks

Since we employ two hash constructions, namely the sponge and the feed-forward sponge structures, for different combinations of r and h , we provide separate explanations for the resistance of each construction against length extension attacks.

1. **SPONGE:** For the sponge construction, resistance to length extension attacks relies on the fact that the inner part of the state remains hidden during the squeezing phase. In particular, only the outer part is released as output, while the c -bit inner part is never exposed to the adversary. As a result, the adversary cannot recover the full internal state, and thus cannot continue the computation to perform a length extension attack.
2. **F-SPONGE:** For the F-sponge construction, resistance to length extension attacks is ensured by a domain separation mechanism. As stated by [11], directly outputting the inner part may enable length-extension attacks. To prevent this, when the last padded message block M_s is XORed into the outer part of the state, a constant $\mathbb{1} = 0^{c-1}\|1$ is simultaneously XORed into the inner part.

Therefore, Mozi is resistant to length extension attacks.

5 Features

The main feature of MOZI is the balance of the security, performance and implement cost.

High security margin. MOZI adopts the sponge construction and Feed-forward Sponge construction which have a lot of security research and rigorous security proofs. The permutation MOZI used also has rigorous security margin, *i.e.*, the limited active S-box numbers. Both of them ensure the security of MOZI.

Performance. MOZI adopts the Feed-forward Sponge construction for larger digest versions and has a relatively large rate ratio compared to classical sponge construction, which makes it to absorb more messages and achieve a high throughput output. It also can be greatly optimized by the vector registers and SIMD instructions in the modern CPU.

Flexibility All operations of the permutation used by MOZI can be implemented in a bit-slice fashion and can be computed in parallel. This makes it flexible to make a balance in the performance and costs.

Innovativeness In this document, we provide MOZI family containing four versions of hash functions and two versions of extendable output functions(XOFs) based on Sponge and Feed-forward Sponge constructions. To support 1024 bits digest, there will be a very large state for popular hash mode constructions such as wide-pipe, sponge etc. To make the state smaller relatively, we choose the Feed-forward Sponge construction for 768 and 1024 bit digests, keeping a good performance and security at the same time. In the aspect of internal permutation, we design bit-slice and parallel friendly components in AES-like style to achieve strong security.

6 Design Rationale

6.1 Choice of the S-box Operation

MOZI adopts a 4-bit S-box, which is the most popular size in modern cryptographic algorithm design. We choose the S-box with the least operations in the algebraic normal form (ANF), *i.e.*, four xors operations plus three and operations and one or operation. The S-box is friendly in bit-slice implementations.

6.2 Choice of Mixcolumn Operation

The MDS matrix has the largest branch number to achieve optimal diffusion in the SPN constructions. We choose a lightweight MDS matrix which has fewer operations and shorter depths at the same time. 8 xors and 4 linear operations make it efficient in bit-sliced implementation.

6.3 Choice of ShiftRow Operation

To strike a balance between cryptographic security and implementation efficiency, the diffusion layer parameters of the proposed algorithm were designed using an automated search technique based on Mixed Integer Linear Programming (MILP). By constructing a precise mathematical model for differential propagation, we systematically evaluated and identified the optimal diffusion layer parameters within a vast parameter space.

State Definition and Truncated Model. The internal state of the algorithm is defined as a $4 \times 4 \times 128$ three-dimensional structure. To effectively manage the solving complexity of the model while accurately characterizing the differential propagation during the automated search, we employed a nibble-oriented truncated differential analysis method. The model constrains and tracks the state variables over a truncated form of 4×128 nibbles.

Search Strategy and Evaluation Criteria. The parameter search and determination process was conducted in two progressive phases: **Fast Diffusion Screening** and **Differential Bound Optimization**.

- **Phase I: Candidate Parameter Screening Based on Full Diffusion Criterion**

The primary design objective of the diffusion layer is to achieve a rapid and thorough avalanche effect. To this end, we established a strict single-nibble difference diffusion constraint:

- *Full Diffusion Condition:* For any non-zero differential input confined to a single nibble in the internal state, the difference must completely diffuse to all nibbles in the state space after 5 rounds of encryption (specifically, after the MixColumns operation in the 5th round).
- By converting the above condition into linear constraints within the MILP model, the solver filtered a large number of candidate parameter sets that satisfy the “5-round full diffusion” property from the global parameter space, thereby constructing an initial candidate parameter pool.

- **Phase II: Optimal Parameter Determination Based on the Number of Active S-boxes**

To provide sufficient security margins against differential cryptanalysis, it is imperative to ensure a high lower bound of active S-boxes after multiple encryption rounds. For the candidate parameter pool generated in Phase I, we performed the following optimization and selection:

- *Long-Round Differential Bound Evaluation:* For each candidate parameter set, the MILP model was utilized to search for its minimum number of active S-boxes over 12 rounds of encryption.
- *Parameter Selection Criterion:* Let N_{active} denote the minimum number of active S-boxes over 12 rounds. We first eliminated all parameter sets with $N_{active} \leq 1024$ to guarantee a baseline security bound. Among the remaining parameters satisfying $N_{active} > 1024$, the set that achieves the global maximum value of N_{active} was selected as the final diffusion layer parameters.

Through the two-phase automated search strategy described above, the finalized diffusion layer parameters achieve both **highly efficient diffusion over a short number of rounds** and **robust security over a long number of rounds**. These parameters not only guarantee a complete state avalanche within 5 rounds but also provide an exceptionally high theoretical upper bound against differential attacks at 12 rounds, laying a solid structural foundation for the overall security of the algorithm.

7 Implementation

7.1 Bit-Slice and Parallel Implementation

MOZI is designed in a bit-slice and parallel fashion. All the operations in its permutation can be implemented in parallel. In terms of state representation,

Each of the four rows contains 128 S-boxes, which can be computed simultaneously. Similarly, operations could be performed in parallel for 128 MixColumns. The following functions are used in the bit-sliced implementation of MOZI.

7.1.1 S-boxes

Let each $x_i (0 \leq i \leq 3)$ denote a 128-bit word, the 4-bit S-box outputs (y_0, y_1, y_2, y_3) are computed by inputs (x_0, x_1, x_2, x_3) as follows:

1. $y_3 = (x_3 \& x_2) \oplus x_1,$
2. $y_1 = (x_2 \mid x_1) \oplus x_0,$
3. $y_0 = (y_3 \& x_0) \oplus x_3,$
4. $y_2 = (y_1 \& x_3) \oplus x_2.$

If the vector registers and SIMD (Single Instruction Multiple Data) instructions are available, 128 Sboxes are computed in parallel. The Sboxes in other row could run concurrently with no mutual interference at the same time, $128 \times 4 = 512$ Sboxes totally.

7.1.2 MixColumns

The bit-slice implementation of MixColumns which is found in [9] is the same as in [6] except the word length. The implementation is shown in Fig. 5 where the inputs/outputs (a, b, c, d) are 128-bit words. The matrix multiplication can be implemented in parallel on each column of the state, so 128 MixColumns could be executed concurrently. The bit-slice implementation with C macro fashion is shown in Listing 1.

Listing 1: bit-slice MDS implementation

```

1 #define MUL(x0, x1, x2, x3, tmp) {
2     tmp = x3; x3 = x2; x2 = x1; x1 = x0; x0 = tmp; /* rotation */
3     x1 ^= x0; /* XOR */
4 }
5
6 #define MDS(x0, x1, x2, x3, x4, x5, x6, x7,
7     x8, x9, xa, xb, xc, xd, xe, xf, tmp) {
8     x8 ^= xc; x9 ^= xd; xa ^= xe; xb ^= xf; /* C ^= D */
9     x0 ^= x4; x1 ^= x5; x2 ^= x6; x3 ^= x7; /* A ^= B */
10    MUL(x4, x5, x6, x7, tmp); /* B = MUL(B) */
11    MUL(xc, xd, xe, xf, tmp); /* D = MUL(D) */
12    x4 ^= x8; x5 ^= x9; x6 ^= xa; x7 ^= xb; /* B ^= C */
13    xc ^= x0; xd ^= x1; xe ^= x2; xf ^= x3; /* D ^= A */
14    MUL(x0, x1, x2, x3, tmp); /* A = MUL(A) */
15    MUL(x0, x1, x2, x3, tmp); /* A = MUL(A) */
16    MUL(x8, x9, xa, xb, tmp); /* C = MUL(C) */
17    MUL(x8, x9, xa, xb, tmp); /* C = MUL(C) */
18    x8 ^= xc; x9 ^= xd; xa ^= xe; xb ^= xf; /* C ^= D */
19    x0 ^= x4; x1 ^= x5; x2 ^= x6; x3 ^= x7; /* A ^= B */

```

```

20     x4 ^= x8; x5 ^= x9; x6 ^= xa; x7 ^= xb; /* B ^= C */
21     xc ^= x0; xd ^= x1; xe ^= x2; xf ^= x3; /* D ^= A */
22 }

```

7.1.3 ShiftRows

In ShiftRow operation, a 128-bit word is shifted circularly to the right by a fixed constant. These constants are shown in Table 3. The implementation of ShiftRow could be accelerated on modern computers with SIMD (Single Instruction Multiple Data) instructions, and the ShiftRows operation can be computed in parallel for 9 lanes with the function as follows:

$$y_i = x_i >>> c_i$$

where c_i is a constant from Table 3.

7.2 SIMD Accelerated Implementation

MOZI is naturally suitable for SIMD acceleration due to its state representation. On mainstream 64-bit computers, a 128-bit word needs two 64-bit registers to store, 128 S-boxes should be computed twice with general-purpose registers, while only one induces with vector registers. It is similar for MixColumns. The only operation need to be changed is the ShiftRows, the C macro code are listed as follows in Listing 2.

Listing 2: SSE implementation of the ShiftRow.

```

1  static inline __m128i rotr128(__m128i x, int n) {
2      __m128i shifted = _mm_srli_epi64(x, n);
3      __m128i shift_left = _mm_slli_epi64(x, 64 - n);
4      __m128i swapped = _mm_shuffle_epi32(shift_left, _MM_SHUFFLE(1, 0, 3,
5          2));
6      __m128i result = _mm_or_si128(shifted, swapped);
7      return result;
8  }
9
10 static inline __m128i rot128bit(__m128i a, int count) {
11     if (count > 64){
12         count = count - 64;
13         a = _mm_shuffle_epi32(a, _MM_SHUFFLE(1, 0, 3, 2));
14     }
15     return rotr128(a, count);
16 }

```

7.3 Resource-constrained Implementation

In resource-constrained environments, the situation is similar to SIMD implementation, since MOZI is designed in a bit-slice fashion. For example, on 32-bit

Table 9: Software performance for 1000-byte messages on the Intel Xeon Gold 6548N platform. Throughput is given in MB/s.

Algorithm	SSE	AVX2	AVX-512
MOZI-384	490.44	488.52	507.15
MOZI-512	443.53	415.33	500.30
MOZI-768	427.21	436.28	459.60
MOZI-1024	332.78	323.54	372.12
SHA3-384	274.12	291.66	402.72
SHA3-512	201.00	207.07	290.51

Table 10: Software performance for 100 MB messages on the Intel Xeon Gold 6548N platform. Throughput is given in MB/s.

Algorithm	SSE	AVX2	AVX-512
MOZI-384	577.74	583.80	598.76
MOZI-512	478.36	448.78	540.27
MOZI-768	476.75	470.53	535.17
MOZI-1024	373.23	361.60	424.57
SHA3-384	316.19	309.67	443.25
SHA3-512	219.00	214.62	307.50

processors, a 128-bit word is represented by four 32-bit words, the S-boxes and Mixcolumns are processed in parallel for these four words and have no interference with each other. The only part that requires extra processing is the shiftrw operation, while there is already much research such as [13] and not a very complex task.

7.4 Software Implementation Performance

We evaluate the software performance of MOZI on a server equipped with Intel(R) Xeon(R) Gold 6548N processors, with a maximum CPU frequency of 4.10 GHz. The operating system is Ubuntu 22.04 LTS. The implementation is single-threaded and compiled by GCC 11.4.0 with the `-O3` optimization option. Three instruction-set variants are tested: SSE, AVX2, and AVX-512. For comparison, we also evaluate SHA3-384 and SHA3-512 using the implementation from XKCP under the same platform and testing methodology.

For each parameter set, two types of messages are tested. The first test uses 1000-byte messages repeated 1,000,000 times (Table 9), and the second test uses 100 MB messages repeated 10 times (Table 10). Therefore, the total amount of processed data is 1000 MB in both tests. The average throughput is computed as the total number of processed bytes divided by the elapsed time.

Table 11: ASIC synthesis results of MOZI on the NanGate 15 nm library.

Algorithm	Structure	Critical Path	Area	Cycles	Throughput
MOZI-384	SPONGE	89.94 ps	13229.80 μm^2	20	711.6 Gbps
MOZI-512	SPONGE	89.94 ps	13229.80 μm^2	20	569.3 Gbps
MOZI-768	F-SPONGE	95.57 ps	16096.64 μm^2	24	530.1 Gbps
MOZI-1024	F-SPONGE	95.57 ps	16096.64 μm^2	24	418.5 Gbps

Table 12: ASIC synthesis results of MOZI on the NanGate 45 nm library.

Algorithm	Structure	Critical Path	Area	Cycles	Throughput
MOZI-384	SPONGE	0.864 ns	58207.72 μm^2	20	74.09 Gbps
MOZI-512	SPONGE	0.864 ns	58207.72 μm^2	20	59.27 Gbps
MOZI-768	F-SPONGE	0.915 ns	70503.30 μm^2	24	55.36 Gbps
MOZI-1024	F-SPONGE	0.915 ns	70503.30 μm^2	24	43.71 Gbps

7.5 Hardware

We evaluate the ASIC performance of MOZI using Synopsys Design Compiler with the NanGate 15 nm and NanGate 45 nm standard-cell libraries. The sponge structure is used for MOZI-384 and MOZI-512, and the F-sponge structure is used for MOZI-768 and MOZI-1024. The throughput is calculated based on the corresponding rate and the number of cycles per message block. The results are shown in Tables 11 and 12.

References

- [1] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A block cipher for low energy. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 411–436. Springer, 2015.
- [2] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The skinny family of block ciphers and its low-latency variant mantis. In *Annual international cryptology conference*, pages 123–153. Springer, 2016.
- [3] Daniel J Bernstein. Cost analysis of hash collisions: Will quantum computers make sharcs obsolete. *SHARCS*, 9:105, 2009.
- [4] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. In *ECRYPT hash workshop*, volume 2007, 2007.
- [5] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN 1998*, pages 163–169. Springer, 1998.

- [6] Anne Canteaut, Sébastien Duval, Gaëtan Leurent, María Naya-Plasencia, Léo Perrin, Thomas Pornin, and André Schrottenloher. Saturnin: a suite of lightweight symmetric algorithms for post-quantum security. 2019.
- [7] André Chailloux, María Naya-Plasencia, and André Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In *ASIACRYPT 2017*, volume 10625, pages 211–240. Springer, 2017.
- [8] Xiaoyang Dong, Jialiang Hua, Siwei Sun, Zheng Li, Xiaoyun Wang, and Lei Hu. Meet-in-the-middle attacks revisited: Key-recovery, collision, and preimage attacks. In *Annual International Cryptology Conference*, pages 278–308. Springer, 2021.
- [9] Sébastien Duval and Gaëtan Leurent. MDS matrices with lightweight circuits. In *IACR Transactions on Symmetric Cryptology*, pages 48–78. Springer, 2018.
- [10] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.
- [11] Chun Guo, Kai Hu, Yanhong Fan, Yong Fu, and Meiqin Wang. Adding feeding forward back to the sponge construction. Cryptology ePrint Archive, Paper 2025/1006, 2025.
- [12] Akinori Hosoyamada and Yu Sasaki. Finding hash collisions with quantum computers by using differential trails with smaller probability than birthday bound. In *EUROCRYPT 2020*, pages 249–279. Springer, 2020.
- [13] Junhao Huang, Alexandre Adomnicăi, Jipeng Zhang, Wangchen Dai, Yao Liu, Ray CC Cheung, Çetin Kaya Koç, and Donglong Chen. Revisiting keccak and dilithium implementations on armv7-m. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):1–24, 2024.
- [14] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 158–188. Springer, 2023.
- [15] Yu Sasaki and Kazumaro Aoki. Finding preimages in full md5 faster than exhaustive search. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 134–152. Springer, 2009.
- [16] André Schrottenloher and Marc Stevens. Simplified mitm modeling for permutations: New (quantum) attacks. In *Annual International Cryptology Conference*, pages 717–747. Springer, 2022.

- [17] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance - application to hash-based signature schemes. Cryptology ePrint Archive, Paper 2025/963, 2025.
- [18] Paul C Van Oorschot and Michael J Wiener. Parallel collision search with cryptanalytic applications. *Journal of cryptology*, 12(1):1–28, 1999.

Supplementary Material

A Table

Table 13: The differential distribution of the MOZI S-box.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	4	0	0	0	0	4	4	4	0	0	0	0	0
2	0	0	4	2	4	2	0	0	0	0	0	2	0	2	0	0
3	0	2	0	0	0	0	0	2	2	0	2	2	2	2	2	0
4	0	4	0	0	0	0	0	4	0	4	0	0	0	0	0	4
5	0	0	0	0	0	4	0	0	4	0	4	0	0	0	0	4
6	0	0	4	2	4	2	0	0	0	0	0	2	0	2	0	0
7	0	2	0	0	0	0	0	2	2	0	2	2	2	2	2	0
8	0	0	0	0	4	0	4	0	0	0	0	0	4	0	4	0
9	0	0	0	0	2	2	2	2	0	0	0	0	2	2	2	2
a	4	0	0	2	0	2	0	0	0	0	0	2	0	2	4	0
b	0	0	0	2	2	0	2	2	2	2	2	0	0	2	0	0
c	0	2	0	2	0	2	0	2	0	2	0	2	0	2	0	2
d	0	2	0	2	2	0	2	0	0	2	0	2	2	0	2	0
e	0	2	4	0	0	0	0	2	0	2	0	0	4	0	0	2
f	0	2	0	0	2	2	2	0	2	0	2	2	0	0	0	2

B Randomness Analysis Result

Table 14: The linear distribution of the MOZI S-box.

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	2	2	0	0	0	0	4	4	2	2	-4	4	0	0
2	0	0	-2	2	0	0	4	0	0	0	2	-2	0	0	0	-4
3	0	0	-2	2	0	0	-4	0	0	0	-2	2	0	0	4	0
4	0	4	2	2	0	0	0	4	2	2	0	4	2	-2	-2	-2
5	0	4	2	2	0	0	0	-4	2	2	4	0	2	-2	2	2
6	0	4	2	-2	0	0	0	0	-2	-2	0	0	-2	2	-2	2
7	0	-4	2	-2	0	0	0	0	2	2	0	0	2	-2	-2	2
8	0	0	2	2	-4	4	0	0	0	0	-2	-2	0	0	0	0
9	0	0	-2	-2	0	0	0	0	4	-4	2	2	0	0	0	0
a	0	0	-2	2	4	4	4	0	0	0	-2	2	0	0	0	4
b	0	0	2	-2	0	0	4	0	0	0	-2	2	4	4	4	0
c	0	0	2	-2	4	0	0	0	-2	2	0	0	-2	-2	2	-2
d	0	0	-2	2	0	-4	0	0	-2	2	0	0	2	2	-2	2
e	0	0	2	2	4	0	0	4	2	-2	0	-4	2	2	2	2
f	0	0	-2	-2	0	4	0	4	-2	2	4	0	2	2	2	2

Table 15: GMT 0005-2021 Randomness Test Results of MOZI-384 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	990	10
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	989	11
<i>PokerTest</i> ($m = 4$)	991	9
<i>PokerTest</i> ($m = 8$)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	991	9
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	984	16
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	989	11
<i>RunsTest</i>	987	13
<i>RunsDistributionTest</i>	990	10
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	985	15
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	995	5
<i>BinaryDerivativeTest</i> ($k = 3$)	986	14
<i>BinaryDerivativeTest</i> ($k = 7$)	982	18
<i>AutocorrelationTest</i> ($d = 1$)	987	13
<i>AutocorrelationTest</i> ($d = 2$)	994	6
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	989	11
<i>MatrixRankTest</i>	992	8
<i>CumulativeTest</i> (Forward)	986	14
<i>CumulativeTest</i> (Backward)	989	11
<i>ApproximateEntropyTest</i> ($m = 2$)	988	12
<i>ApproximateEntropyTest</i> ($m = 5$)	983	17
<i>LinearComplexityTest</i> ($m = 500$)	990	10
<i>LinearComplexityTest</i> ($m = 1000$)	990	10
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	985	15
<i>DiscreteFourierTransformTest</i>	989	11

Table 16: GMT 0005-2021 Randomness Test Results of MOZI-384 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	989	11
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	991	9
<i>PokerTest</i> ($m = 4$)	991	9
<i>PokerTest</i> ($m = 8$)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	992	8
<i>RunsTest</i>	989	11
<i>RunsDistributionTest</i>	994	6
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	993	7
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	991	9
<i>BinaryDerivativeTest</i> ($k = 3$)	987	13
<i>BinaryDerivativeTest</i> ($k = 7$)	984	16
<i>AutocorrelationTest</i> ($d = 1$)	989	11
<i>AutocorrelationTest</i> ($d = 2$)	989	11
<i>AutocorrelationTest</i> ($d = 8$)	986	14
<i>AutocorrelationTest</i> ($d = 16$)	991	9
<i>MatrixRankTest</i>	995	5
<i>CumulativeTest</i> (Forward)	992	8
<i>CumulativeTest</i> (Backward)	989	11
<i>ApproximateEntropyTest</i> ($m = 2$)	986	14
<i>ApproximateEntropyTest</i> ($m = 5$)	995	5
<i>LinearComplexityTest</i> ($m = 500$)	989	11
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	988	12
<i>DiscreteFourierTransformTest</i>	985	15

Table 17: GMT 0005-2021 Randomness Test Results of MOZI-384 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	992	8
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	987	13
<i>PokerTest</i> ($m = 4$)	992	8
<i>PokerTest</i> ($m = 8$)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	993	7
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	987	13
<i>RunsTest</i>	991	9
<i>RunsDistributionTest</i>	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	995	5
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	992	8
<i>BinaryDerivativeTest</i> ($k = 3$)	989	11
<i>BinaryDerivativeTest</i> ($k = 7$)	991	9
<i>AutocorrelationTest</i> ($d = 1$)	991	9
<i>AutocorrelationTest</i> ($d = 2$)	994	6
<i>AutocorrelationTest</i> ($d = 8$)	993	7
<i>AutocorrelationTest</i> ($d = 16$)	995	5
<i>MatrixRankTest</i>	987	13
<i>CumulativeTest</i> (Forward)	994	6
<i>CumulativeTest</i> (Backward)	990	10
<i>ApproximateEntropyTest</i> ($m = 2$)	993	7
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	996	4
<i>LinearComplexityTest</i> ($m = 1000$)	993	7
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	982	18
<i>DiscreteFourierTransformTest</i>	991	9

Table 18: GMT 0005-2021 Randomness Test Results of MOZI-512 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	991	9
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	990	10
<i>PokerTest</i> ($m = 4$)	994	6
<i>PokerTest</i> ($m = 8$)	987	13
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	993	7
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	991	9
<i>RunsTest</i>	989	11
<i>RunsDistributionTest</i>	991	9
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	986	14
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	991	9
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	989	11
<i>AutocorrelationTest</i> ($d = 1$)	989	11
<i>AutocorrelationTest</i> ($d = 2$)	996	4
<i>AutocorrelationTest</i> ($d = 8$)	993	7
<i>AutocorrelationTest</i> ($d = 16$)	989	11
<i>MatrixRankTest</i>	993	7
<i>CumulativeTest</i> (Forward)	985	15
<i>CumulativeTest</i> (Backward)	991	9
<i>ApproximateEntropyTest</i> ($m = 2$)	995	5
<i>ApproximateEntropyTest</i> ($m = 5$)	987	13
<i>LinearComplexityTest</i> ($m = 500$)	991	9
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	990	10
<i>DiscreteFourierTransformTest</i>	987	13

Table 19: GMT 0005-2021 Randomness Test Results of MOZI-512 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	991	9
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	992	8
<i>PokerTest</i> ($m = 4$)	995	5
<i>PokerTest</i> ($m = 8$)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	990	10
<i>RunsTest</i>	982	18
<i>RunsDistributionTest</i>	992	8
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	995	5
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	989	11
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	990	10
<i>AutocorrelationTest</i> ($d = 1$)	982	18
<i>AutocorrelationTest</i> ($d = 2$)	994	6
<i>AutocorrelationTest</i> ($d = 8$)	988	12
<i>AutocorrelationTest</i> ($d = 16$)	990	10
<i>MatrixRankTest</i>	987	13
<i>CumulativeTest</i> (Forward)	990	10
<i>CumulativeTest</i> (Backward)	989	11
<i>ApproximateEntropyTest</i> ($m = 2$)	992	8
<i>ApproximateEntropyTest</i> ($m = 5$)	989	11
<i>LinearComplexityTest</i> ($m = 500$)	989	11
<i>LinearComplexityTest</i> ($m = 1000$)	990	10
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	989	11
<i>DiscreteFourierTransformTest</i>	990	10

Table 20: GMT 0005-2021 Randomness Test Results of MOZI-512 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	988	12
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	986	14
<i>PokerTest</i> ($m = 4$)	992	8
<i>PokerTest</i> ($m = 8$)	987	13
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	994	6
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	994	6
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	991	9
<i>RunsTest</i>	988	12
<i>RunsDistributionTest</i>	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	993	7
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	988	12
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	992	8
<i>AutocorrelationTest</i> ($d = 1$)	988	12
<i>AutocorrelationTest</i> ($d = 2$)	990	10
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	993	7
<i>MatrixRankTest</i>	990	10
<i>CumulativeTest</i> (Forward)	988	12
<i>CumulativeTest</i> (Backward)	986	14
<i>ApproximateEntropyTest</i> ($m = 2$)	994	6
<i>ApproximateEntropyTest</i> ($m = 5$)	994	6
<i>LinearComplexityTest</i> ($m = 500$)	995	5
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	988	12
<i>DiscreteFourierTransformTest</i>	988	12

Table 21: GMT 0005-2021 Randomness Test Results of MOZI-768 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	990	10
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	989	11
<i>PokerTest</i> ($m = 4$)	992	8
<i>PokerTest</i> ($m = 8$)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	989	11
<i>RunsTest</i>	990	10
<i>RunsDistributionTest</i>	991	9
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	987	13
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	989	11
<i>BinaryDerivativeTest</i> ($k = 3$)	991	9
<i>BinaryDerivativeTest</i> ($k = 7$)	994	6
<i>AutocorrelationTest</i> ($d = 1$)	990	10
<i>AutocorrelationTest</i> ($d = 2$)	993	7
<i>AutocorrelationTest</i> ($d = 8$)	990	10
<i>AutocorrelationTest</i> ($d = 16$)	992	8
<i>MatrixRankTest</i>	986	14
<i>CumulativeTest</i> (Forward)	992	8
<i>CumulativeTest</i> (Backward)	993	7
<i>ApproximateEntropyTest</i> ($m = 2$)	992	8
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	988	12
<i>LinearComplexityTest</i> ($m = 1000$)	985	15
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	992	8
<i>DiscreteFourierTransformTest</i>	987	13

Table 22: GMT 0005-2021 Randomness Test Results of MOZI-768 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	988	12
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	988	12
<i>PokerTest</i> ($m = 4$)	982	18
<i>PokerTest</i> ($m = 8$)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	993	7
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	991	9
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	989	11
<i>RunsTest</i>	985	15
<i>RunsDistributionTest</i>	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	986	14
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	992	8
<i>BinaryDerivativeTest</i> ($k = 3$)	994	6
<i>BinaryDerivativeTest</i> ($k = 7$)	990	10
<i>AutocorrelationTest</i> ($d = 1$)	985	15
<i>AutocorrelationTest</i> ($d = 2$)	989	11
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	996	4
<i>MatrixRankTest</i>	989	11
<i>CumulativeTest</i> (Forward)	987	13
<i>CumulativeTest</i> (Backward)	988	12
<i>ApproximateEntropyTest</i> ($m = 2$)	992	8
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	991	9
<i>LinearComplexityTest</i> ($m = 1000$)	995	5
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	985	15
<i>DiscreteFourierTransformTest</i>	982	18

Table 23: GMT 0005-2021 Randomness Test Results of MOZI-768 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	994	6
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	988	12
<i>PokerTest</i> ($m = 4$)	993	7
<i>PokerTest</i> ($m = 8$)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	996	4
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	997	3
<i>RunsTest</i>	989	11
<i>RunsDistributionTest</i>	987	13
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	989	11
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	994	6
<i>BinaryDerivativeTest</i> ($k = 3$)	991	9
<i>BinaryDerivativeTest</i> ($k = 7$)	985	15
<i>AutocorrelationTest</i> ($d = 1$)	989	11
<i>AutocorrelationTest</i> ($d = 2$)	997	3
<i>AutocorrelationTest</i> ($d = 8$)	986	14
<i>AutocorrelationTest</i> ($d = 16$)	994	6
<i>MatrixRankTest</i>	990	10
<i>CumulativeTest</i> (Forward)	992	8
<i>CumulativeTest</i> (Backward)	991	9
<i>ApproximateEntropyTest</i> ($m = 2$)	995	5
<i>ApproximateEntropyTest</i> ($m = 5$)	995	5
<i>LinearComplexityTest</i> ($m = 500$)	991	9
<i>LinearComplexityTest</i> ($m = 1000$)	988	12
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	991	9
<i>DiscreteFourierTransformTest</i>	989	11

Table 24: GMT 0005-2021 Randomness Test Results of MOZI-1024 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	993	7
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	990	10
<i>PokerTest</i> ($m = 4$)	991	9
<i>PokerTest</i> ($m = 8$)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	992	8
<i>RunsTest</i>	992	8
<i>RunsDistributionTest</i>	993	7
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	981	19
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	990	10
<i>AutocorrelationTest</i> ($d = 1$)	992	8
<i>AutocorrelationTest</i> ($d = 2$)	989	11
<i>AutocorrelationTest</i> ($d = 8$)	988	12
<i>AutocorrelationTest</i> ($d = 16$)	989	11
<i>MatrixRankTest</i>	991	9
<i>CumulativeTest</i> (Forward)	993	7
<i>CumulativeTest</i> (Backward)	995	5
<i>ApproximateEntropyTest</i> ($m = 2$)	990	10
<i>ApproximateEntropyTest</i> ($m = 5$)	997	3
<i>LinearComplexityTest</i> ($m = 500$)	986	14
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	990	10
<i>DiscreteFourierTransformTest</i>	988	12

Table 25: GMT 0005-2021 Randomness Test Results of MOZI-1024 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	989	11
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	991	9
<i>PokerTest</i> ($m = 4$)	989	11
<i>PokerTest</i> ($m = 8$)	994	6
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	991	9
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	989	11
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	990	10
<i>RunsTest</i>	997	3
<i>RunsDistributionTest</i>	987	13
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	994	6
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	991	9
<i>BinaryDerivativeTest</i> ($k = 3$)	984	16
<i>BinaryDerivativeTest</i> ($k = 7$)	991	9
<i>AutocorrelationTest</i> ($d = 1$)	997	3
<i>AutocorrelationTest</i> ($d = 2$)	991	9
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	992	8
<i>MatrixRankTest</i>	993	7
<i>CumulativeTest</i> (Forward)	988	12
<i>CumulativeTest</i> (Backward)	988	12
<i>ApproximateEntropyTest</i> ($m = 2$)	991	9
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	990	10
<i>LinearComplexityTest</i> ($m = 1000$)	983	17
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	987	13
<i>DiscreteFourierTransformTest</i>	990	10